# Geometric Control and Trajectory Tracking of a Simulated Quadrotor: Modelling and Experimental Evaluation

Lab 3: Unified Individual Report

Junxi Li
Student ID: 24020036033 Group 6
Ocean University of China

***Abstract*—This report examines time-parameterised circular tracking, including the position, heading and timing relationships that connect a reference to rotor commands. It presents the ROS rotation conversions, quadrotor actuation model, geometric controller, and trajectory-tracking experiment completed by Group 6. The analysis derives the rank loss caused by a rotor-direction exchange and formulates quadratic body-drag compensation in world coordinates. The system uses a minimum-jerk transfer, a hold, and a smooth angular-rate ramp to join the initial pose to a 5 m-radius circle at 2.5 m/s. Over 1,498 samples in the steady-state window, the three-dimensional position RMS error is 0.04102 m, the distance-to-circle RMS error is 0.002448 m, and the yaw RMS error is 3.925 degrees. Tangential displacement contributes 85.96% of position mean-square error, compared with 0.39% from radial projection. The error decomposition highlights reference timing and desired attitude-rate tracking as priorities for further analysis.**

## I. Introduction

Lab 3 connects attitude representation, vehicle dynamics, control allocation, reference generation, and feedback through a simulated quadrotor flight. The report follows the exercise sequence: ROS quaternion handling in Deliverable 1, rotor allocation and drag compensation in Deliverable 2, and system implementation and trajectory tracking in Deliverable 3. The modelling and control methods are grounded in the exercise specification and geometric tracking formulation [1], [3].

The four members of Group 6 worked collaboratively, with responsibilities distributed across modelling and coordinate conventions, geometric control and allocation, ROS and simulator integration, and reference trajectory design and experimental validation. My role was reference trajectory design and experimental validation. I focus particularly on the smooth start-up reference, the definition and interpretation of tracking error, and the recorded flight results.

## II. Deliverable 1: rotation representations in ROS

A ROS quaternion message stores orientation as four fields. A `tf2::Quaternion` provides quaternion operations, and an Eigen matrix supports the vector calculations in geometric feedback. The conversions below use ROS 1 interfaces and their corresponding conversion headers [2].

### A. A1: message to an existing tf2 quaternion

For an incoming `quat_msg` and an already declared `quat_tf`, the required line is

```
tf2::fromMsg(quat_msg, quat_tf);
```

It maps the message fields into the mathematical quaternion object. Validate incoming components and normalise a non-unit quaternion before using it as a rotation.

### B. A2: tf2 quaternion to a new message

The new message is declared and initialised in one line:

```
geometry_msgs::Quaternion quat_msg = tf2::toMsg(quat_tf);
```

This is useful when the reference generator publishes the desired heading.

### C. A3: scalar component

The scalar component is obtained using

```
quat_tf.getW();
```

The alternative accessor `w()` is also available. For a unit quaternion representing angle $\alpha$, the scalar part is $w = \cos(\alpha/2)$; it is neither the norm of the quaternion nor the angle itself.

### D. B1: extracting yaw

One function call gives the yaw angle in radians:

```
double yaw = tf2::getYaw(quat_t);
```

Yaw is a wrapped angular coordinate, so evaluation wraps the difference between measured and desired yaw across the $[-\pi, \pi]$ branch cut.

### E. B2: message to an Eigen rotation matrix

The conversion uses an intermediate quaternion:

```
Eigen::Quaterniond q_eigen;
tf2::fromMsg(quat_msg, q_eigen);
Eigen::Matrix3d R =
    q_eigen.toRotationMatrix();
```

The conversion uses `tf2::fromMsg` and Eigen's `toRotationMatrix()`. Validate the message components and quaternion norm before conversion, normalising a valid non-unit quaternion. ROS fields are ordered $(x, y, z, w)$,

while Eigen's four-scalar constructor expects $(w, x, y, z)$; the conversion function handles this ordering directly.

The required headers for these answers are

```
#include <geometry_msgs/Quaternion.h>
#include <tf2/LinearMath/Quaternion.h>
#include <tf2/utils.h>
#include <tf2_geometry_msgs/tf2_geometry_msgs.h>
#include <tf2_eigen/tf2_eigen.h>
#include <Eigen/Geometry>
```

## III. Deliverable 2: actuation and drag

### A. A: why a propeller exchange changes controllability

The exercise numbering is defined in Fig. 1. It is distinct from the simulator's rotor ordering used later. Let $a = d/\sqrt{2}$, where $d$ is the distance from the centre of mass to a rotor. The rotor positions in body coordinates are

$$\begin{aligned} \boldsymbol{\rho}_1 &= (-a, a, 0)^T, \quad \boldsymbol{\rho}_2 = (-a, -a, 0)^T, \\ \boldsymbol{\rho}_3 &= (a, -a, 0)^T, \quad \boldsymbol{\rho}_4 = (a, a, 0)^T. \end{aligned} \tag{1}$$

For squared speed magnitude $u_i = \omega_i^2 \geq 0$, lift is $c_f u_i \mathbf{e}_3$, with $c_f > 0$. Its moment about the centre of mass is

$$\boldsymbol{\rho}_i \times c_f u_i \mathbf{e}_3 = c_f u_i (\rho_{iy}, -\rho_{ix}, 0)^T. \tag{2}$$

Let $c_d > 0$ be the reaction-torque coefficient. With $+z$ pointing out of the top-view drawing, a counter-clockwise rotor produces a clockwise reaction moment on the body. Thus a CCW rotor contributes $-c_d u_i$ to $M_z$, and a CW rotor contributes $+c_d u_i$.

Collect the wrench and inputs as $\boldsymbol{w} = (f, M_x, M_y, M_z)^T$ and $\boldsymbol{u} = (u_1, u_2, u_3, u_4)^T$. In the conventional layout, $\boldsymbol{w} = F_a \boldsymbol{u}$, where

$$F_a = \begin{bmatrix} c_f & c_f & c_f & c_f \\ a c_f & -a c_f & -a c_f & a c_f \\ a c_f & a c_f & -a c_f & -a c_f \\ -c_d & c_d & -c_d & c_d \end{bmatrix}. \tag{3}$$

Divide the rows by $c_f$, $a c_f$, $a c_f$ and $c_d$, respectively. The four resulting rows are nonzero and mutually orthogonal, so they are independent. Equivalently,

$$\det F_a = 16 a^2 c_f^3 c_d \neq 0, \qquad \operatorname{rank} F_a = 4. \tag{4}$$

Consequently, thrust and the three body moments can be adjusted independently in a local operating region that respects rotor limits.

After the exchange, only the reaction-torque row changes:

$$F_b = \begin{bmatrix} c_f & c_f & c_f & c_f \\ a c_f & -a c_f & -a c_f & a c_f \\ a c_f & a c_f & -a c_f & -a c_f \\ -c_d & c_d & c_d & -c_d \end{bmatrix}. \tag{5}$$

The last row is now $-c_d/(a c_f)$ times the second row. The first three rows remain independent, giving

$$\operatorname{rank} F_b = 3, \qquad M_z = -\frac{c_d}{a c_f} M_x. \tag{6}$$

This row dependence constrains every attainable wrench. For example, maintaining level position while accelerating yaw requires $M_x = 0$ and, for the symmetric rigid-body model with pure yaw motion, $M_z = J_{zz} \ddot{\psi}_d \neq 0$. Equation (6) forbids that combination. The exchanged configuration can still execute special motions, but it cannot independently follow an arbitrary prescribed $(x, y, z, \psi)$ trajectory. Rank four by itself also does not guarantee that every wrench is physically feasible: nonnegative inputs and actuator limits remain relevant.

### B. B: compensating quadratic body drag

Let $\mathbf{R} \in SO(3)$ map body vectors into the world frame, and let $\boldsymbol{v}$ be world velocity. The body velocity and specified drag force are

$$\begin{aligned} \boldsymbol{v}^b &= \mathbf{R}^T \boldsymbol{v}, \qquad D = \operatorname{diag}(0.1, 0.1, 0.2), \\ \boldsymbol{F}_D^b &= -D\big(\boldsymbol{v}^b \odot |\boldsymbol{v}^b|\big), \\ \boldsymbol{F}_D^w &= \mathbf{R} \boldsymbol{F}_D^b. \end{aligned} \tag{7}$$

Absolute value and multiplication in the middle expression are componentwise. Its dissipative sign can be checked directly:

$$(\boldsymbol{v}^b)^T \boldsymbol{F}_D^b = -0.1 |v_x^b|^3 - 0.1 |v_y^b|^3 - 0.2 |v_z^b|^3 \leq 0. \tag{8}$$

Combining body-frame drag directly with world-frame feedback would be incorrect unless the two frames happened to coincide.

Using the laboratory's upward $z$ convention, translational dynamics are

$$m \dot{\boldsymbol{v}} = -mg \mathbf{e}_3 + f \mathbf{R} \mathbf{e}_3 + \boldsymbol{F}_D^w, \qquad \mathbf{e}_3 = (0, 0, 1)^T. \tag{9}$$

For $\boldsymbol{e}_x = \boldsymbol{x} - \boldsymbol{x}_d$ and $\boldsymbol{e}_v = \boldsymbol{v} - \dot{\boldsymbol{x}}_d$, a drag-compensated desired force is

$$\mathbf{A} = -k_x \boldsymbol{e}_x - k_v \boldsymbol{e}_v + mg \mathbf{e}_3 + m \ddot{\boldsymbol{x}}_d - \widehat{\boldsymbol{F}}_D^w. \tag{10}$$

The estimated external force is subtracted because it already acts on the vehicle. As a sign check, $\mathbf{R} = I$ and $\boldsymbol{v} = (2, 0, 0)^T$ m/s give $\widehat{\boldsymbol{F}}_D^w = (-0.4, 0, 0)^T$ N; compensation therefore adds $0.4$ N in the positive $x$ direction.

The force direction and requested yaw define a desired attitude:

$$\begin{aligned} \boldsymbol{b}_{3d} &= \mathbf{A} / \|\mathbf{A}\|, \qquad \boldsymbol{b}_{1c} = (\cos \psi_d, \sin \psi_d, 0)^T, \\ \boldsymbol{b}_{2d} &= \frac{\boldsymbol{b}_{3d} \times \boldsymbol{b}_{1c}}{\|\boldsymbol{b}_{3d} \times \boldsymbol{b}_{1c}\|}, \quad \boldsymbol{b}_{1d} = \boldsymbol{b}_{2d} \times \boldsymbol{b}_{3d}, \\ \mathbf{R}_d &= [\boldsymbol{b}_{1d} \ \boldsymbol{b}_{2d} \ \boldsymbol{b}_{3d}], \qquad f = \mathbf{A}^T \mathbf{R} \mathbf{e}_3. \end{aligned} \tag{11}$$

These expressions require a nonzero force and a nondegenerate heading cross-product. Compensation can change both the thrust magnitude and the desired tilt; it is not simply an additive scalar thrust offset.

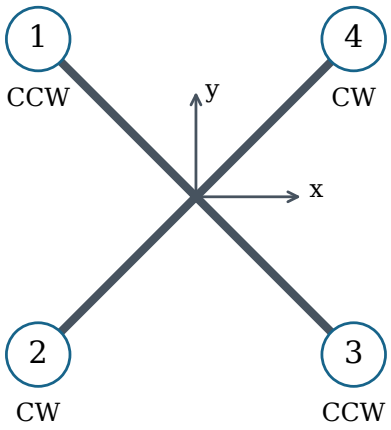
For completeness, the geometric attitude errors of [3] are

$$\begin{aligned} \boldsymbol{e}_R &= \tfrac{1}{2} (\mathbf{R}_d^T \mathbf{R} - \mathbf{R}^T \mathbf{R}_d)^\vee, \\ \boldsymbol{e}_\Omega &= \boldsymbol{\Omega} - \mathbf{R}^T \mathbf{R}_d \boldsymbol{\Omega}_d. \end{aligned} \tag{12}$$

Here $\widehat{\boldsymbol{a}} \boldsymbol{b} = \boldsymbol{a} \times \boldsymbol{b}$, and $\vee$ is the inverse of this skew-matrix map. The full moment law is

$$\begin{aligned} \boldsymbol{M} = & -k_R \boldsymbol{e}_R - k_\Omega \boldsymbol{e}_\Omega + \boldsymbol{\Omega} \times J \boldsymbol{\Omega} \\ & - J\big(\widehat{\boldsymbol{\Omega}} \mathbf{R}^T \mathbf{R}_d \boldsymbol{\Omega}_d - \mathbf{R}^T \mathbf{R}_d \dot{\boldsymbol{\Omega}}_d\big). \end{aligned} \tag{13}$$

(a) Independent wrench: rank 4




(b) Roll-yaw coupling: rank 3

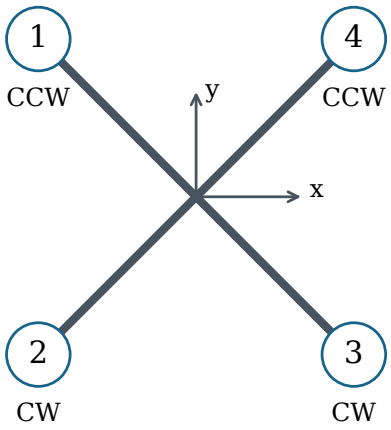



Fig. 1. Quadrotor rotor configurations considered in Deliverable 2A. Rotor centres stay fixed; the spin directions at positions 3 and 4 are exchanged. Axes are body axes in the top view, with $+z$ out of the page. This numbering is used only in Deliverable 2A.

The specified drag model acts at the centre of mass and introduces no additional moment. Combining the translational equations gives the useful error identity

$$m\dot{\boldsymbol{e}}_v + k_v\boldsymbol{e}_v + k_x\boldsymbol{e}_x = (f\mathbf{R}\mathbf{e}_3 - \mathbf{A}) + (\boldsymbol{F}_D^w - \widehat{\boldsymbol{F}}_D^w). \tag{14}$$

With exact drag estimation and ideal force realisation, the right-hand side vanishes. Positive gains then yield the desired second-order error dynamics $m\ddot{\boldsymbol{e}}_x + k_v\dot{\boldsymbol{e}}_x + k_x\boldsymbol{e}_x = 0$. In practice, force-direction error, delay, drag-model error and actuator constraints contribute to the residual.

## IV. Deliverable 3: Implementation and Trajectory Tracking

### A. What the executed controller computes

Our implementation separates ROS communication from geometric control calculations [4]. The node receives desired position, velocity and acceleration on `/desired_state`, measured pose and twist on `/current_state`, and publishes four signed speeds on `/rotor_speed_cmds`.

The state callback normalises a valid quaternion and forms $\mathbf{R}$. TESSE reports angular velocity in world coordinates for this interface, so the controller uses $\boldsymbol{\Omega} = \mathbf{R}^T\boldsymbol{\omega}^w$. This transformation is essential because the attitude feedback and rigid-body moment equation use body angular velocity. The callback also rejects nonfinite data and near-zero quaternions. The timer publishes only after both state and reference have arrived and their receipt ages are within the configured limits.

The recorded controller uses the force construction in Eq. (10) with quadratic-drag compensation disabled, and implements the simplified moment law

$$\boldsymbol{M} = -k_R\boldsymbol{e}_R - k_\Omega\boldsymbol{\Omega} + \boldsymbol{\Omega}\times J\boldsymbol{\Omega}. \tag{15}$$

Although the trajectory message contains desired yaw rate, the callback stores only desired yaw. Desired angular velocity and angular acceleration feedforward from Eq. (13) are therefore omitted; their role in the measured heading error is discussed in Section VI.

For the simulator rotor order, the centres are $(a,a)$, $(-a,a)$, $(-a,-a)$ and $(a,-a)$, and positive reaction-torque signs alternate as $(+,-,+,-)$. The implemented allocation is

$$F_{\text{sim}} = \begin{bmatrix} c_f & c_f & c_f & c_f \\ ac_f & ac_f & -ac_f & -ac_f \\ -ac_f & ac_f & ac_f & -ac_f \\ c_d & -c_d & c_d & -c_d \end{bmatrix}. \tag{16}$$

Here the signed input is $s_i = \omega_i|\omega_i|$, so the inverse map is

$$\boldsymbol{s} = F_{\text{sim}}^{-1}\boldsymbol{w}, \qquad \omega_i = \operatorname{sgn}(s_i)\sqrt{|s_i|}. \tag{17}$$

This differs from the nonnegative squared magnitudes used in the rotor layout analysis of Deliverable 2A. The laboratory simulator accepts signed commands; the controller does not apply motor-speed or slew-rate limits.

Table I distinguishes configured rates from observed records. The mass, gravity, arm length and inertia are controller-model parameters; the simulator's internal rigid-body properties are not included in the telemetry. The experiment was recorded on 20 October 2026 in the official TESSE simulator with ROS Noetic on x86_64 Ubuntu using hardware-assisted rendering. The run script selects `gains_gpu.yaml`; the launch-file default is `gains_unity.yaml`.

### B. The circular reference and its derivatives

Once the ramp is complete, the reference is

$$\begin{aligned} \boldsymbol{x}_d(t) &= (r\sin\theta, r\cos\theta, h)^T, \\ \theta(t) &= \omega t, \qquad \psi_d(t) = -\theta(t), \\ r &= 5\ \text{m}, \quad h = 2\ \text{m}, \quad \omega = 0.5\ \text{rad/s}. \end{aligned} \tag{18}$$

TABLE I
RECORDED CONFIGURATION AND CONTROLLER CONSTANTS

| Quantity | Value |
|---|---|
| Position / velocity gains | $k_x = 14$, $k_v = 6$ |
| Attitude / rate gains | $k_R = 20$, $k_\Omega = 3$ |
| Control timer setting | 200 Hz |
| Reference publisher setting | 100 Hz |
| Observed state records | 50 Hz |
| State / reference receipt timeout | 1.0 s / 1.0 s |
| Controller model mass / gravity | 1.0 kg / 9.81 m/s$^2$ |
| Arm length / model inertia | 0.3 m / $I_3$ kg m$^2$ |
| Lift / reaction coefficients | $10^{-3}$ / $10^{-5}$ (SI) |

The period is $2\pi/\omega = 4\pi$ s, speed is $r\omega = 2.5$ m/s and centripetal acceleration is $r\omega^2 = 1.25$ m/s$^2$. The horizontal heading $(\cos\psi_d, \sin\psi_d)$ is aligned with the direction of motion $(\cos\theta, -\sin\theta)$. For a changing phase rate, the correct derivatives are

$$\begin{aligned} \dot{\boldsymbol{x}}_d &= (r\cos\theta\dot\theta, -r\sin\theta\dot\theta, 0)^T, \\ \ddot{x}_d &= -r\sin\theta\dot\theta^2 + r\cos\theta\ddot\theta, \\ \ddot{y}_d &= -r\cos\theta\dot\theta^2 - r\sin\theta\ddot\theta, \\ \ddot{z}_d &= 0. \end{aligned} \tag{19}$$

Keeping the $\ddot\theta$ terms is necessary during the rate ramp; using only the constant-speed acceleration would make the published position and acceleration inconsistent.

### C. Minimum-jerk transition from the spawn pose

The initial measured position is approximately $(0, 0, 0.7000)$ m, while the circle starts at $(0, 5, 2)$ m. To connect these poses smoothly, the generator reads the initial pose and uses a six-second minimum-jerk transfer. For transfer time $\tau \in [0, T]$, $T = 6$ s and $u = \tau/T$, define

$$\begin{aligned} S(u) &= 10u^3 - 15u^4 + 6u^5, \\ \dot S &= 30u^2(1-u)^2/T, \\ \ddot S &= 60u(1-u)(1-2u)/T^2. \end{aligned} \tag{20}$$

The position is $\boldsymbol{x}_0 + S(\boldsymbol{x}_c - \boldsymbol{x}_0)$, with velocity and acceleration obtained by differentiating the same polynomial. Both vanish at the endpoints, providing continuous position, velocity and acceleration at the transfer joins. Yaw follows the shortest wrapped rotation from the initial heading to zero using the same blend.

The code defines circle time with a 12 s lead. Consequently, transfer runs from $t = -12$ to $-6$ s and hold from $-6$ to $-3$ s. The six-second ramp then spans $t = -3$ to 3 s. With $u = (t+3)/6$ in the ramp,

$$\begin{aligned} \theta &= \omega T_r(u^3 - \tfrac{1}{2}u^4), \quad T_r = 6 \text{ s}, \\ \dot\theta &= \omega(3u^2 - 2u^3), \\ \ddot\theta &= 6\omega u(1-u)/T_r. \end{aligned} \tag{21}$$

At $t = 3$ s, phase is 1.5 rad, rate is 0.5 rad/s, and phase acceleration is zero. All match $\theta = t/2$ and its derivatives, so the standard circle follows without a position, velocity or acceleration jump. Figure 2 shows recorded position during this transition and the analytic rate specified by the code. The rate panel plots the derivative of the reference phase used by the trajectory generator.

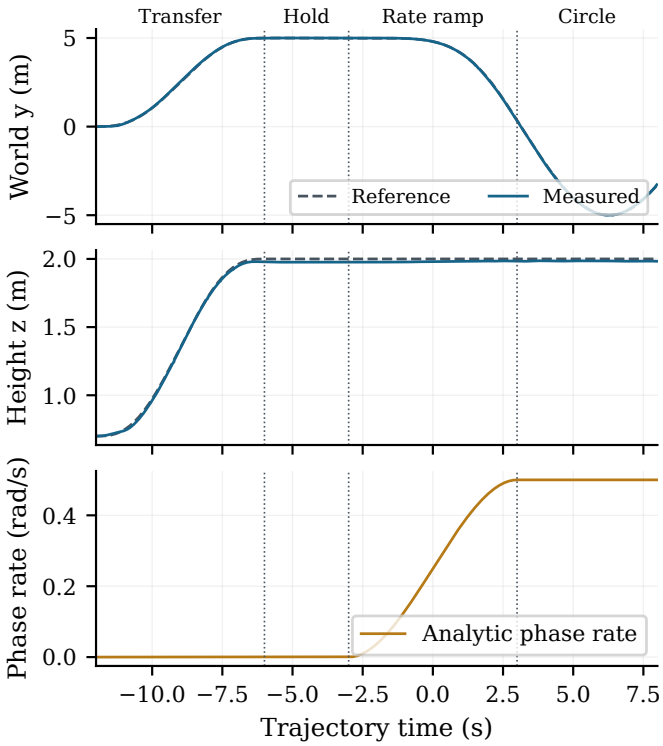



Fig. 2. Start-up reference and recorded motion. The upper panels use the raw CSV; the lower panel is computed from Eq. (21). Vertical guides separate transfer, hold, rate ramp and constant-rate circle. The negative time labels are the generator's trajectory clock.

## V. EXPERIMENTAL EVALUATION OF TRAJECTORY TRACKING

### A. Dataset and evaluation window

State timestamps increase strictly, with a median interval of 0.0200 s; the trajectory-time column spans $-12.00$ to 55.08 s. These values are simulation times rather than MP4 playback times.

Steady-state metrics use $t \geq 8\pi$ s, from 25.14 to 55.08 s, comprising 1,498 records after the rate ramp. RMS values are calculated from the recorded samples. Figure 3 shows the full-flight motion and steady path; the time-domain error measures complement this geometric view.

For each state callback, the monitor saves the latest available reference and rotor command alongside the state. State timestamps are retained, while the reference and command source timestamps are not synchronised in this log. The recording contains 99 repeated consecutive trajectory-time entries despite strictly increasing state timestamps. The sample-wise errors below therefore use the logged state/reference pairs; timestamp-aligned evaluation would interpolate the reference at each state timestamp.

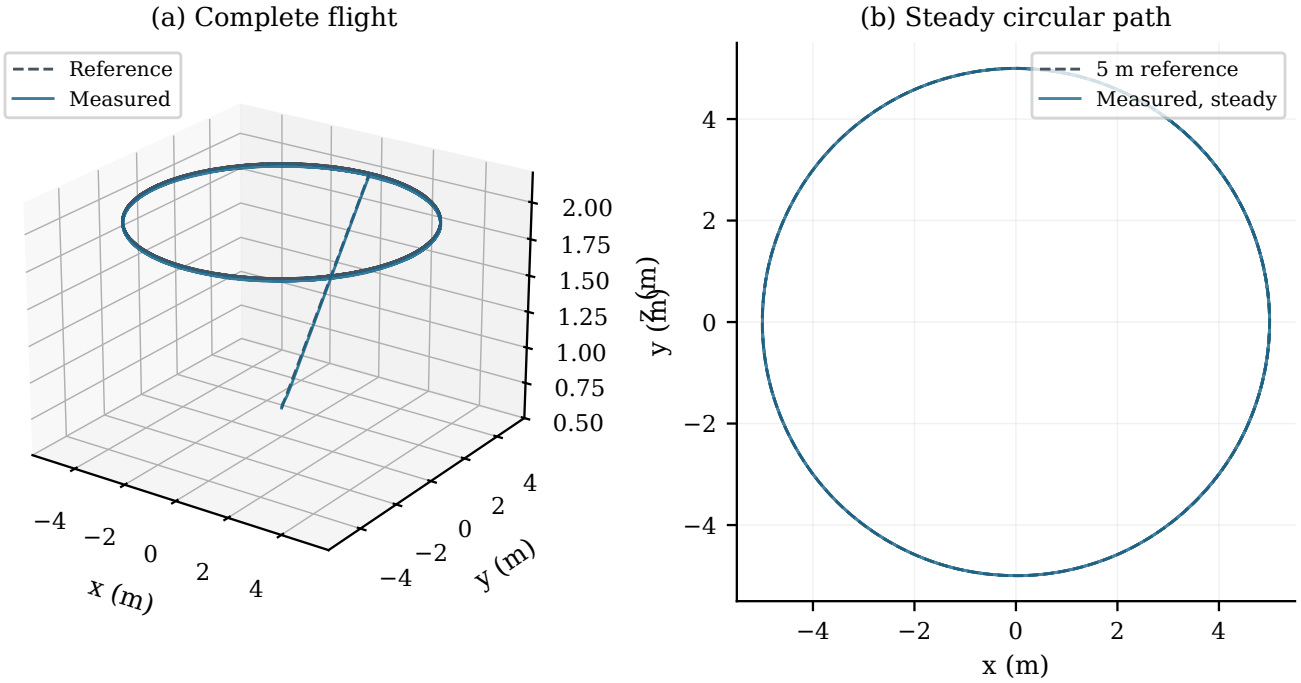



Fig. 3. Recorded quadrotor trajectory and prescribed circular reference. (a) Transfer from the initial height to the circular path; the vertical axis range differs from the horizontal ranges. (b) Equal-scale XY view of the steady window. The close path overlap shows geometric agreement; the time-domain results quantify position at the recorded reference times.

### *B. Separating shape error from along-path error*

For each recorded pair, define $\boldsymbol{e} = \boldsymbol{x} - \boldsymbol{x}_d$ and

$$E_{\mathrm{RMS}} = \sqrt{\frac{1}{N}\sum_{j=1}^{N} \|\boldsymbol{e}_j\|^2}, \qquad E_{\max} = \max_j \|\boldsymbol{e}_j\|. \tag{22}$$

Distance from the ideal circular cylinder is measured by

$$e_{\mathrm{circle}} = \sqrt{x^2 + y^2} - r. \tag{23}$$

It ignores phase and height. To decompose the actual position error, define radial and tangent unit vectors at the desired phase:

$$\boldsymbol{n} = (\sin\theta, \cos\theta, 0)^T, \qquad \boldsymbol{q} = (\cos\theta, -\sin\theta, 0)^T. \tag{24}$$

Then

$$\begin{gathered} e_r = \boldsymbol{n}^T\boldsymbol{e}, \quad e_t = \boldsymbol{q}^T\boldsymbol{e}, \quad e_z = z - z_d, \\ \|\boldsymbol{e}\|^2 = e_r^2 + e_t^2 + e_z^2. \end{gathered} \tag{25}$$

The radial projection $e_r$ differs from $e_{\mathrm{circle}}$. If the actual planar radius is $\rho$ and its phase offset is $\delta$, then $e_r = \rho\cos\delta - r$, whereas $e_{\mathrm{circle}} = \rho - r$. The former is part of the orthogonal position-error decomposition; the latter measures distance from the ideal circle.

Yaw error is calculated using

$$e_\psi = \mathrm{atan2}\big(\sin(\psi - \psi_d), \cos(\psi - \psi_d)\big). \tag{26}$$

The CSV records yaw, world angular velocity and position. It supports yaw and position error analysis; full $SO(3)$ attitude error and roll/pitch error curves require complete orientation measurements.

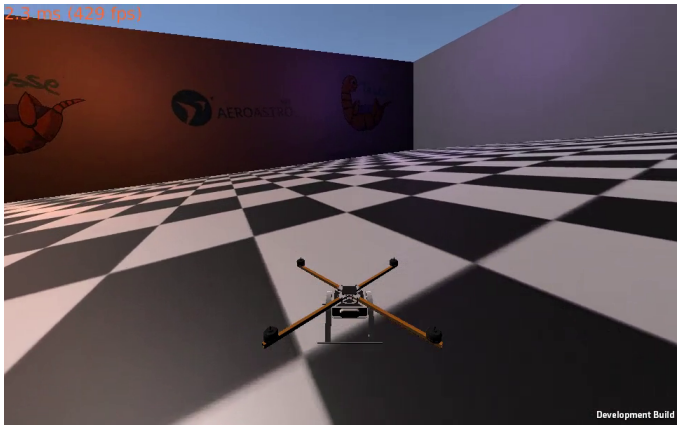
TABLE II
STEADY-STATE RESULTS, $25.14 \le t \le 55.08$ S, $N = 1498$

| Measure | Result |
|---|---|
| Three-dimensional position RMS | 0.041024 m |
| Maximum three-dimensional error | 0.102512 m |
| $x$ / $y$ RMS | 0.026085 / 0.027800 m |
| Distance-to-circle RMS | 0.002448 m |
| Radial-projection RMS | 0.002572 m |
| Tangential-projection RMS | 0.038035 m |
| Vertical RMS | 0.015155 m |
| Mean vertical error | −0.015052 m |
| Measured height range | 1.981336–1.988438 m |
| Wrapped yaw RMS | 3.9252° |
| Mean wrapped yaw error | +3.9210° |
| Observed angular travel | 2.38170 laps |

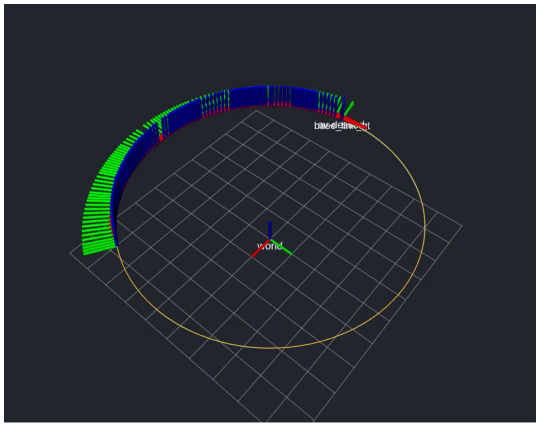
### *C. Position and heading results*

Table II reports the steady-window metrics. Observed laps are calculated by unwrapping $\mathrm{atan2}(x, y)$ and dividing the net angular change by $2\pi$. The steady interval includes more than two revolutions. The radial distance RMS is only 2.448 mm, while the three-dimensional RMS is 41.024 mm. This difference is explained by Eq. (25): radial projection, tangential projection and vertical error account for 0.393%, 85.960% and 13.647% of total position mean-square error, respectively.

Figure 5 makes the different scales visible. Height remains below the 2 m target throughout the steady window, with




(a) Unity video, playback time 00:35



(b) RViz video, playback time 00:55

Fig. 4. Frames extracted from the recorded Unity and RViz demonstrations. The Unity view shows the simulation environment; the RViz view shows the circular reference and body-frame visualisation. Video links are provided in Section VIII.

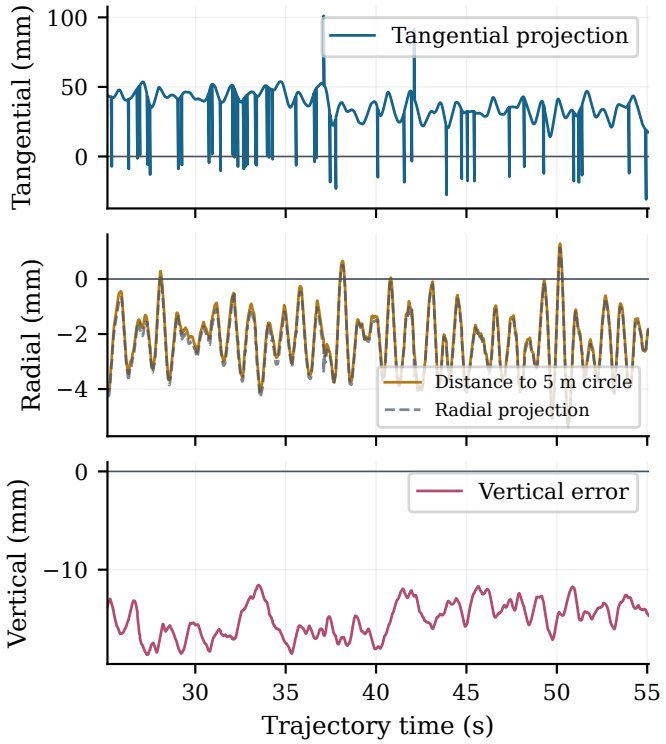



Fig. 5. Steady-window position errors from the recorded state/reference pairs. The panels use different vertical scales, with all errors expressed in millimetres. The middle panel distinguishes radial projection from distance to the ideal circle. Tangential error dominates the exact orthogonal decomposition.

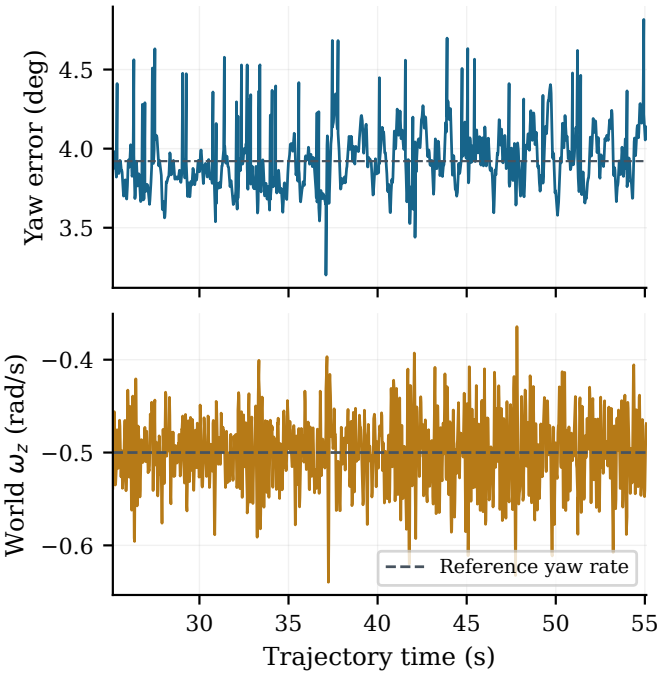



Fig. 6. Wrapped yaw error (top) and recorded world $z$ angular velocity (bottom). Dashed lines indicate the mean yaw error and desired yaw rate, respectively. Both panels use the same steady window.

an average bias of $-15.052$ mm. Tangential error averages $+36.232$ mm in the direction of reference travel. Relative to the stored reference, the vehicle is predominantly ahead along the path. At the nominal speed, this mean tangential offset corresponds to 14.49 ms of travel time, a useful scale for assessing timestamp alignment.

The heading plot in Fig. 6 shows a persistent positive yaw offset with RMS $3.9252°$. The world $z$ angular rate is close to the commanded yaw rate of $-0.5$ rad/s; a small rate error can coexist with a nonzero angular offset. Because the vehicle is tilted while turning, the world $z$ component differs from the body-rate error used by the controller.

### *D. Rotor commands and video evidence*

Figure 7 shows every recorded rotor command in a two-second window. Across the complete steady-state evaluation window, signed commands range from $-135.281$ to $144.868$ rad/s, and $28.60\%$ of the recorded rotor entries are negative. These are signed simulator commands rather than measured motor speeds, and follow the inverse map in Eq. (17). The

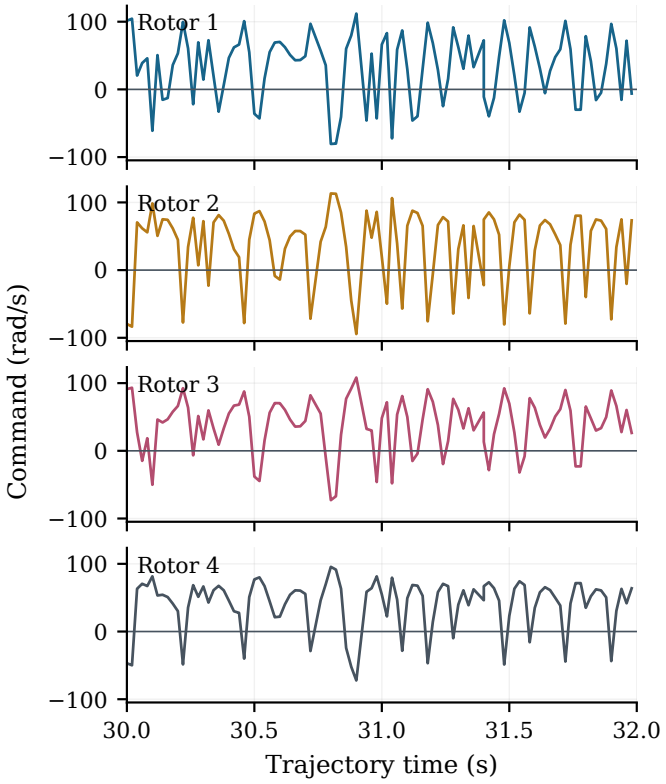



Fig. 7. Signed rotor commands over $30 \leq t \leq 32$ s, at the recorded 50 Hz state-callback cadence. Axes have identical limits across the four panels. Negative values are preserved as recorded and follow the TESSE signed-speed convention.

telemetry samples at 50 Hz, while the configured controller timer runs at 200 Hz.

The Unity recording is 69.30 s at $960 \times 600$ pixels; the RViz recording is 79.13 s at $900 \times 710$ pixels. Both are 30 frames/s. The frames in Fig. 4 were extracted directly at playback times 35 s and 55 s, respectively. They show the simulated scene and the circular RViz reference with the body-frame trail; each time denotes playback within its own recording.

## VI. Individual Discussion

### A. Reference timing and along-track error

The path follows the circle closely, while tangential RMS error (38.035 mm) is much larger than radial-projection RMS error (2.572 mm). This pattern directs attention to the time relationship between state and reference. At 2.5 m/s, a 10 ms mismatch corresponds to 25 mm of along-path travel and a 20 ms mismatch to 50 mm, comparable to the 36.232 mm mean tangential offset. The logger pairs each state with the latest reference; 99 repeated trajectory-time values and missing reference source timestamps make timing alignment a key part of interpreting the measured error.

In a subsequent evaluation, I would retain source timestamps for every state, reference and command, then interpolate the analytic reference to each state timestamp. Comparing the logged pairs with timestamp-aligned results would help distinguish reference-sampling effects from feedback response. This comparison would quantify how reference timing contributes to the reported along-path tracking error.

### B. Interpretation of the heading offset

The simplified attitude law damps measured body angular velocity toward zero while the desired heading continues to rotate. In a near-level, single-axis approximation with $\dot{\psi} \simeq \dot{\psi}_d = -0.5$ rad/s, the steady moment balance is

$$0 \simeq -k_R \sin e_\psi - k_\Omega \dot{\psi}_d, \quad e_\psi \simeq \arcsin\left(\frac{3(0.5)}{20}\right) = 4.30^\circ. \tag{27}$$

This simplified estimate gives $e_\psi \simeq 4.30^\circ$, close in sign and magnitude to the measured $+3.921^\circ$ mean offset. Tilt, rotational coupling, simulator dynamics and asynchronous logging also shape the observed response. Desired body angular velocity derived consistently from $\mathbf{R}_d(t)$ would account for the rotating reference directly and is a targeted next change to evaluate before increasing $k_R$.

### C. Continuous reference design and further validation

The minimum-jerk transfer sets velocity and acceleration to zero at both endpoints. The angular-rate ramp starts with zero phase rate and acceleration and joins the constant-speed circle at its prescribed rate and acceleration. Together, these segments maintain continuity in desired position, velocity and acceleration, giving the controller a smooth target through take-off and into circular flight.

To quantify the effect of this start-up policy, I would compare it with a direct start while holding initial pose, controller gains, simulator build and evaluation interval fixed. Repeated trials would permit comparison of peak transient error and steady-state error under each policy.

### D. Practical limits and priorities

The steady flight has a mean vertical error of $-15.052$ mm, alongside rapid variation in signed rotor commands. The altitude bias motivates checking vertical force calibration and attitude tracking. The simulator's reversible-input convention also differs from physical motor operation; future allocation should enforce rotor-speed, slew-rate and thrust constraints while evaluating the achievable wrench. My priorities are timestamp-aligned analysis, desired attitude-rate feedforward, and a controlled gain study, followed by validation with motor dynamics included.

## VII. Conclusion

ROS quaternion conversions provide the orientation representations used in the controller. The rotor-allocation derivation shows how exchanging spin directions reduces wrench rank from four to three and couples roll with yaw; the drag analysis expresses body-frame resistance in world coordinates for force compensation. In Deliverable 3, Group 6 implemented geometric feedback and used a minimum-jerk transfer, a hold, and a rate ramp to join take-off to the circular reference.

Over 1,498 steady-state samples, the recorded flight has a 41.024 mm three-dimensional position RMS error, a 2.448 mm distance-to-circle RMS error, and a $3.925^\circ$ yaw RMS error. Tangential displacement contributes 85.96% of position mean-square error, identifying reference timing as an important evaluation target. Timestamp alignment and desired attitude-rate feedforward are the next steps for interpreting and improving the tracking response.

## VIII. Source code and recordings

**Individual D1 -- ROS conversions**
`https://github.com/Moln-ya/vnav-personal/blob/main/lab3/deliverable_1.txt`
**Individual D2 -- modelling solutions**
`https://github.com/Moln-ya/vnav-personal/blob/main/lab3/deliverable_2.pdf`
**Team Lab3 -- controller and experiments**
`https://github.com/chixin114514/2026Fall_TeamFullcans/tree/main/lab3`
**Flight data -- recorded run**
`https://github.com/chixin114514/2026Fall_TeamFullcans/tree/main/lab3/report/runs/unity-gpu-20261020`
**Unity recording**
`https://github.com/chixin114514/2026Fall_TeamFullcans/blob/main/lab3/report/video/unity_gpu_sim.mp4`
**RViz recording -- new team repository**
`https://github.com/chixin114514/2026Fall_TeamFullcans/blob/main/lab3/report/video/rviz_gpu.mp4`
**RViz source of Fig. 4 -- personal archive**
`https://github.com/Moln-ya/vnav-personal/blob/main/lab3/report/video/rviz_body_frames.mp4`
**Team video-path notes**
`https://github.com/chixin114514/2026Fall_TeamFullcans/blob/main/lab3/rviz_drone_flight.txt`

## References


[1] MIT VNAV, "Lab 3: Exercises," 2023 course materials. https://vnav.mit.edu/labs_2023/lab3/exercises.html.

[2] ROS geometry2 maintainers, "tf2, tf2_geometry_msgs and tf2_eigen," ROS 1 Noetic source API. https://github.com/ros/geometry2/tree/noetic-devel.

[3] T. Lee, M. Leok, and N. H. McClamroch, "Geometric tracking control of a quadrotor UAV on SE(3)," in Proc. 49th IEEE Conference on Decision and Control, 2010, pp. 5420-5425, doi: 10.1109/CDC.2010.5717652.

[4] Group 6, "Lab 3 controller, flight telemetry and recordings," 20 October 2026. https://github.com/chixin114514/2026Fall_TeamFullcans/tree/main/lab3